

Database Integration Guide

Wind Data Insight Pro

Version: 1.0

Date: November 27, 2025

Author: Development Team

Status: Implementation Guide

Table of Contents

1. Overview
2. Architecture
3. File Structure Changes
4. Integration Steps
5. Code Examples
6. Implementation Checklist
7. Testing Strategy
8. Backward Compatibility

1. Overview

This document provides a comprehensive guide for integrating database features from the New_features module into the main Wind Data Insight Pro application. The integration enables SQLite-based storage, automatic turbine clustering, and efficient data management for wind farm projects.

Key Features

- SQLite database backend for project data storage
- Automatic detection and clustering of turbine data
 - Dynamic table creation per project
- Turbine-specific data retrieval and filtering
 - Database viewer for data inspection

Database Components

Component	Purpose
database_manager.py	SQLite operations and project management
schema_detector.py	Auto-detect turbine columns and cluster data
db_viewer.py	GUI for database inspection
test_db_flow.py	Complete workflow testing

2. Architecture

The database integration follows a layered architecture approach, integrating seamlessly with the existing MVC pattern of the application.

Integration Points

Layer	Component	Changes Required
Model	DataManager	Add database backend and clustering
Controller	ProjectController	Store metadata with DB references
Controller	FileHandler	Add auto-clustering on file load
View	CentralWidget	Add turbine selector UI
View	MenuBar	Add database menu options
Config	Configuration	Add database settings

3. File Structure Changes

The following files need to be relocated from New_features/ to the main application structure:

Source	Destination	Reason
New_features/database_manager.py	models/database_manager.py	Data model layer
New_features/schema_detector.py	utils/schema_detector.py	Utility function
New_features/db_viewer.py	views/dialogs/db_viewer_dialog.py	UI component

Note: After moving files, update all import statements to reflect the new locations.

4. Integration Steps

Step 1: DataManager Enhancement

- Add database_manager instance
- Implement load_data_with_clustering() method
- Implement save_to_database() method
 - Add turbine_clusters property
- Add get_turbine_data() method

Step 2: ProjectController Update

- Add database fields to project structure
- Update _update_project_metadata() to save DB info
- Update _set_current_project() to restore from DB
 - Add database path management

Step 3: FileHandler Integration

- Add clustering on file load
- Implement clustering result dialog
 - Add database save option
- Update data loading workflow

Step 4: UI Components

- Add turbine selector to CentralWidget
 - Implement turbine filtering
- Add database viewer button
- Update data table display

Step 5: Menu Integration

- Add Database menu to MenuBar
- Implement database actions
 - Add keyboard shortcuts

Step 6: Configuration

- Add database settings to config.json
- Update ConfigurationManager
 - Add settings dialog options

5. Code Examples

5.1 DataManager Enhancement

```
def load_data_with_clustering(self, data): from utils.schema_detector import
SchemaDetector self.data = data detector = SchemaDetector() wtg_column =
detector.detect_wtg_column(data) if wtg_column: self.turbine_clusters =
detector.cluster_by_turbine(data, wtg_column) self.metadata['wtg_column'] =
wtg_column self.metadata['turbine_count'] = len(self.turbine_clusters) return
wtg_column, self.turbine_clusters
```

5.2 Turbine Selector UI

```
def on_turbine_selected(self, wtg_id: str): if wtg_id == "All Turbines": data =
self.data_manager.data else: data = self.data_manager.get_turbine_data(wtg_id)
if data is not None: self.data_table_module.update_data_table(data)
```

5.3 Enhanced Project Structure

```
{ "metadata": { "name": "Wind Farm Project", "database_enabled": true,
"database_path": "Project/WindFarm.db", "turbine_count": 25, "turbine_ids":
["WTG01", "WTG02", ...] }, "data": { "storage_type": "database",
"database_project_id": 1, "wtg_column": "Turbine" } }
```

6. Implementation Checklist

Phase 1: File Structure (Day 1)

- Move database_manager.py to models/
 - Move schema_detector.py to utils/
- Move db_viewer.py to views/dialogs/
 - Update all import statements

Phase 2: Core Integration (Day 2-3)

- Add database support to DataManager
- Implement load_data_with_clustering()
 - Implement save_to_database()
 - Test clustering functionality

Phase 3: Project Controller (Day 4)

- Update project structure
- Implement database save/restore
 - Test project lifecycle

Phase 4: File Handler (Day 5)

- Add clustering on file load
- Implement clustering dialog
 - Test file import workflow

Phase 5: UI Components (Day 6-7)

- Add turbine selector
- Implement data filtering
 - Add database viewer
 - Test UI interactions

Phase 6: Menu Integration (Day 8)

- Add Database menu
- Implement menu actions
 - Test shortcuts

Phase 7: Configuration (Day 9)

- Add database config
- Update settings dialog
 - Test configuration

Phase 8: Testing & Polish (Day 10)

- Unit tests
- Integration tests
- Performance testing
- Documentation

7. Testing Strategy

Unit Tests

- test_schema_detector.py - WTG detection and clustering
 - test_database_manager.py - Database operations
- test_data_manager_integration.py - Integration methods

Integration Tests

- test_full_workflow.py - Complete data flow
- test_ui_integration.py - UI component interactions
 - test_project_lifecycle.py - Save/load cycles

Performance Tests

- Large dataset handling (100k+ rows)
 - Multiple turbines (50+ turbines)
 - Database query performance
 - Memory usage monitoring

8. Backward Compatibility

To ensure existing projects continue to work, the integration maintains backward compatibility with JSON-based storage.

Compatibility Strategy

- Check database_enabled flag in project metadata
- Fall back to JSON storage if flag is False or missing
- Provide migration tool for converting old projects
- Support both storage methods simultaneously
 - Maintain existing project file format

Migration Process

```
def migrate_to_database(self, project_data: dict, project_path: str):
    active_dataset = project_data.get("data", {}).get("active_dataset") if
    active_dataset and active_dataset.get("data"): data =
    pd.DataFrame(active_dataset["data"]) db_path = project_path.replace(".wdip",
    ".db") # Create database and cluster data data_manager = DataManager()
    data_manager.enable_database(db_path) wtg_column, clusters =
    data_manager.load_data_with_clustering(data) data_manager.save_to_database(None,
    project_data["metadata"]["name"]) # Update project structure
    project_data["metadata"]["database_enabled"] = True
    project_data["metadata"]["database_path"] = db_path return True return False
```